

Self-play Learning for Two-player Score-based Board Games

Progress Report

Cristina Cutajar
University of Malta
cristina.cutajar.20@um.edu.mt

ABSTRACT

The aim of this project is to determine which reinforcement learning algorithm is the most ideal for the game of Jaipur, a two-player score-based board-game. This game is challenging because its state/s are partially observable and its actions are stochastic. Furthermore, the game has a very large action space. Reference is made to different research that was conducted on similar games as, until now, there is no publicly available research on how to best go about implementing reinforcement learning on the game of Jaipur. The game as well as a reinforcement learning environment have been created and tested thoroughly. The observation space and action space are being designed and when an ideal solution for these is achieved, reinforcement learning algorithms will be applied and evaluated with the environment.

1. INTRODUCTION

For this project, the two-player score-based board-game named “Jaipur” was chosen. This board game consists of two players where both players trade cards in order for them to get points and try to win the game.

1.1 Problem Definition

The game of Jaipur is a turn-based card game where both players will take turns to perform an action each until the game ends. During game-play, the players’ hand cards and the marketplace cards are visible to both players, however, the players’ hand cards are only visible to the respective player. Moreover, the deck cards are shuffled and flipped over so none of the players can be aware of which card will be dealt next. Therefore, these rules make the game’s environment partially observable and stochastic.

1.2 Motivation

The game of Jaipur was selected for this project due to numerous factors which make the problem challenging.

The first factor would be that the game contains a dynamic environment as different opponents have different play styles and so the agent must adjust its policy according to the current opponent’s play style.

Moreover, the game of Jaipur has a partially observable environment as not all the game information is visible. The opponent’s initial hand cards aren’t visible, and the agent must

take note of what cards the opponent is taking from the marketplace in order to get an idea of what hand cards the opponent has. Apart from this, the deck cards are flipped over which prevents both players from seeing what card will be placed into the marketplace.

In addition to the above, the environment contains stochastic actions as performing the same action leads to different outcomes. For example, when the agent chooses to take a card from the marketplace, the card will be replaced with a random card which may be better and hence, the opponent will then be given the opportunity to take it.

Furthermore, the game has a large action space, with 25,456 possible actions when exchanging cards from the marketplace. Apart from these, there are also another 6 possible actions when taking one card from the marketplace, a possible action of taking all the camels present in the marketplace and 6 possible actions of selling goods. As a comparison, chess only has 4,672 possible moves [11].

Lastly, the game contains actions which provide:

- Immediate rewards and good long-term consequences. For example selling the first card/s of a goods’ type gives the highest points for that card’s type.
- Immediate rewards but bad long-term consequences. For example selling 1 or 2 cards instead of waiting to sell 3, 4 or preferably 5 cards at the same time and getting the bonus points.
- Delayed higher future reward. For example waiting to sell 3, 4 or preferably 5 cards of the same type in order to get the bonus token points.
- Delayed lower future reward. For example waiting to collect 3, 4 or 5 cards but the opponent sells cards of the same type beforehand and takes all the remaining tokens or the game finishes.

Therefore, the agent must learn when to take the immediate reward and when to wait for a more long-term reward based on the game state.

1.3 Aims and Objectives

The aim of this project is to develop an intelligent agent that is able to play the two-player score-based board game Jaipur

well. In order for this to be achieved, the following objectives have been identified:

1. Implement the rules of the game,
2. Develop a reinforcement learning environment to be able to test different algorithms,
3. Determine which observation space and action space can be used to capture the state information of the game,
4. Evaluate different reinforcement learning algorithms with different reward functions to test the agent's performance.

2. BACKGROUND RESEARCH

In the game of Jaipur, there are 7 different card types: Diamonds, Gold, Silver, Silk, Spice, Leather and Camel, which together make up a total of 55 cards. Both players start off with 5 random cards each whilst the marketplace is initialized with 3 Camel cards and 2 random cards allocated to it from the shuffled deck. Camel card types are stored as the player's herd and should be visible to everyone, whilst the other card types (Goods cards) are stored as the player's hand and are only visible to the respective player. The players can have as many Camel type cards as they want in their herd, however, they can never have more than 7 Goods type cards in their hand.

For every Goods card type, there are a set of tokens each with different values, highest value first. Apart from this, there are also bonus tokens for when a player sells 3, 4 or 5 cards respectively and each of these sets of tokens are shuffled. At the end of the game, the player with the most Camel cards gets the Camel Bonus token which is worth 5 points.

The players will take turns of performing one action until the game ends. There are four actions that a player can choose from. These actions are: Taking one Goods card from the marketplace, taking all the Camel cards from the marketplace, swapping between 2 to 5 cards from the marketplace with cards from the player's hand and/or herd or selling cards of the same type in exchange for tokens.

The game ends when either all tokens of three goods have been taken or when there are no more cards in the deck to replace the cards taken from the marketplace. The player with the most points wins the round. A full game can be played with 1 to 3 rounds.

2.1 Reinforcement Learning

Reinforcement learning refers to the training performed by an agent in order to select the best actions which should be taken to maximise the return and the sum of all rewards [12]. Reinforcement learning occurs during interaction with the environment. Through trial and error, the agent learns a policy by making use of the Reward Hypothesis whilst performing exploration or exploitation [16]. As shown in Figure 1, the agent makes use of observation states from the environment, applies an action to the current state and gets the reward. From the reward, the agent will learn which actions are best to take in each state.

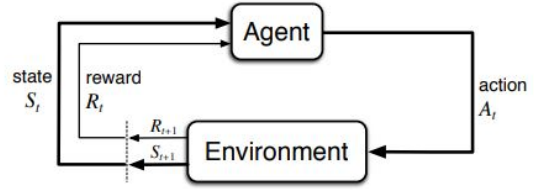


Figure 1. Reinforcement Learning Interaction Loop [12]

2.2 Deep Learning

Deep learning utilizes a range of deep neural network architectures to perform either supervised or unsupervised learning with a large amount of hand-labelled training data. By doing so, the model learns to extract high-level features directly from raw sensory data such as images [8].

2.3 Deep Reinforcement Learning

Deep reinforcement learning is a type of learning which combines reinforcement learning with deep learning to allow high-dimensional interactive learning [10]. Deep reinforcement learning algorithms can take in very large input of unstructured data and learn which actions to perform to maximise reward.

Deep Q-Network (DQN) is an example of a deep reinforcement learning algorithm which is a combination of deep learning and Q-learning. It makes use of a deep neural network to calculate the Q-value of each action from the given state and then chooses the action with the highest Q-value. Because of this, this algorithm can only be used in environments with discrete action spaces. However, it proves to be beneficial in a high-dimensional environment, where the combination of states and actions are too large, as this algorithm is still able to achieve stable learning since it derives the approximation of the Q-value function instead of keeping track of all the solutions. By doing so, this off-policy algorithm managed to overcome the instability of the deep reinforcement learning process which was previously thought to be impossible to overcome. [10]

The DQN algorithm has a tendency to overestimate action values [10] and so, the Double DQN algorithm was developed to address the overestimation. This is done by evaluating the policy's value with the use of a second set of weights [14]. Moreover, the Dueling DDQN algorithm was later on introduced from Double DQN to be able to standardise action values. This algorithm learns which states are ideal without having to evaluate each action from each state [15].

Monte-Carlo Tree Search (MCTS) is an algorithm which is often improved with deep learning to be implemented in games with large action spaces. This algorithm performs best-first search on stochastic simulations in order to find the best solution. This algorithm will first create a tree of all the possible future game states and then it will perform four steps in order to find the best action. These four steps consist of selection, expansion, simulation and backpropagation. The selection step will loop until an action, which balances out exploitation and exploration, is found. Afterwards, expansion will be

performed for the simulated game state to be added as a new tree node. The simulation step will then be performed for the game to be played with actions chosen at random. Finally, the algorithm will then perform backpropagation to update the tree nodes with the information gathered from the simulated game. The ideal action will then be performed in the actual game. [2]

3. LITERATURE REVIEW

Research was performed on the different reinforcement learning algorithms and techniques used on similar games as there is no publicly available studies on how to go about solving this problem with the game of Jaipur.

Similarly to the game of Jaipur, the game of DouDizhu also has a large action space. Zha et al. [21] proposed the application of an algorithm called DouZero which makes use of parallel agents, action space encoding and deep neural networks to enhance the traditional Monte-Carlo methods. This algorithm is based on sampling which allows for more data to be generated per second since more complex neural architectures can be used.

The board game “Go” has a series of AI which were developed to play the full game. The series consists of different versions which are AlphaGo, AlphaGo Zero and AlphaZero. AlphaGo, which was developed in 2015, managed to beat the European Go champion with a score of 5 to 0 [19]. This was very impressive especially since it was the first time that an AI won against a professional player whilst playing the full game [4], [19]. AlphaGo Zero was then developed to reach superhuman performance without the need for prior training data of expert moves [4], [19]. Later on, AlphaZero was developed which, not only masters the game of Go, but also the board games Chess and Shogi. All the AlphaGo versions contain the Monte-Carlo Tree Search algorithm which was improved with deep learning to allow the search to be able to handle such a large complexity. This algorithm is repeated thousands of times in order to achieve a final decision. [4], [9], [19]

Fujita [4] proposed the game AI AlphaDDA which is based on AlphaZero. Similar to AlphaZero, this algorithm also makes use of deep neural networks with Monte-Carlo Tree Search. Unlike AlphaZero, AlphaDDA will sometimes purposefully select actions which are not the most ideal. This allows the algorithm to adapt dynamically according to its opponent by estimating the game state value and adjusting according to it. This in turn allows the algorithm to provide a challenging but reasonable environment for the player as it gives the player the opportunity to win.

Ghory [5], Konen [7] and Wiering et al. [17] talk about the use of the Temporal Difference algorithm for board game learning. This technique makes use of neural networks with either one or two layers for the agent to be able to generalize. For learning to occur, the neural networks are trained with a combination of the TD algorithm, to approximate the game theoretic function, and the backpropagation method.

Furthermore, research was performed on how large-scale action spaces were encoded. Dulac-Arnold et al. [3] proposed a new policy architecture, called Wolpertinger architecture,

which is able to perform efficiently with a large action space. This was accomplished by embedding prior knowledge about the actions in a continuous space for the agent to be able to generalise the actions in logarithmic time. This architecture makes use of multi-layer neural networks as an actor-critic framework as well as Deep Deterministic Policy Gradient to train the policy. The policy proposed by Dulac-Arnold et al. [3] generates an approximate nearest-neighbor search from the continuous action space to achieve a set of the ideal discrete actions. By doing so, the policy will evade the heavy cost of having to evaluate all the actions. The policy will then select the ideal action from the set of discrete actions either by choosing the highest valued action, in respect to a cost function, or by choosing the closest action from the set.

Moreover, Kanervisto et al. [6] and Zahavy et al. [20] talk about how action space shaping, to make learning easier, can be achieved by the following: representing different actions into a single action, converting continuous actions into discrete actions, or by removing un-promising actions from the action space.

Yao et al. [18] proposed a hybrid reinforcement learning technique which is able to learn with the huge discrete action space of the game of Axie Infinity. The use of a Markov Decision Process is made to encode the action space. The applied reinforcement learning technique generates a small set of actions, from the large-scale action set of all the possible actions, and then selects the most optimal action from the subset. The small action set is generated with actions whose results are similar to the target action. This was accomplished with the use of a Euclidean distance function. The action with the highest Q-value from the closest actions will be chosen by the policy.

4. PROPOSED SOLUTION AND METHODOLOGY

The programming language Python was chosen in order to implement and achieve all the objectives of this project.

4.1 Objective 1: Implement the rules of the game

The first objective is to implement the entire game and all its rules from scratch. In order to achieve this, four classes were implemented: Player class, Card class, Token class and the Jaipur class. The classes and all of their variables and functions are shown in Figure 2.

The Player class initializes all the needed variables to store the relevant data about each player such as the player's ID, herd and hand size, bonus points, cards, Seal of Excellence count and opponent points and camels.

The Card class contains four functions which are used to create the cards according to the rules of the game, shuffle the cards and deal them. The game's cards include: 11 Camel cards, 6 Diamond cards, 6 Gold cards, 6 Silver cards, 8 Silk cards, 8 Spice cards and 10 Leather cards.

The Token class is used to initialise all the tokens according to the game of Jaipur. The game consists of 1 Camel token, with a value of 5 Rupees, and 38 Goods tokens. These Goods tokens consist of 5 Diamond tokens, 5 Gold tokens, 5 Silver tokens, 7 Silk tokens, 7 Spice tokens and 9 Leather tokens. All the sets of Goods tokens have specific values, and they are all

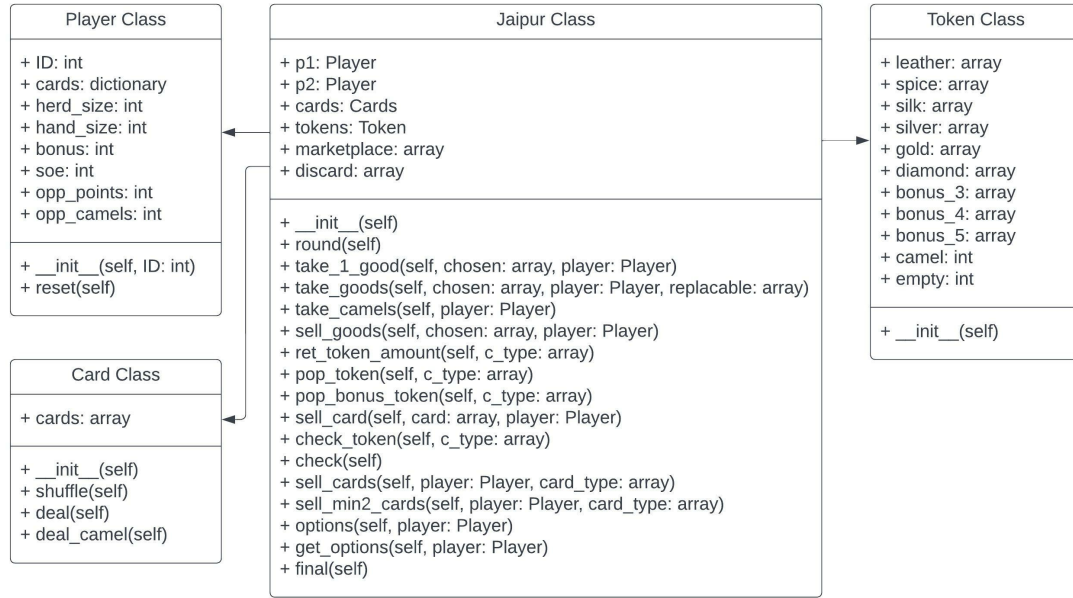


Figure 2. UML diagram representing the architecture of the implementation.

sorted in descending order, highest values first. The game also consists of 7 bonus tokens for when a player sells 3 cards at a time, 6 bonus tokens for when a player sells 4 cards at a time and 5 bonus tokens for when a player sells 5 cards at a time. Unlike the other sets of tokens, which are sorted in descending order, these sets of bonus tokens are all individually shuffled.

In the actual board game, there are also 3 Seals of Excellence tokens, which are to be used when playing 2 or 3 rounds as one game, to keep track of who won the rounds. However, this information will be kept in the player object, instead of as a token in the Token class.

The Jaipur class contains all the necessary functions to perform the game. This class first initializes two player objects by calling the Player class. Then, the game's "round" function will first create the cards and tokens by calling the Card and Token classes. It will also create the marketplace and fill it with the initial cards according to the game rules, which are 3 Camel cards and 2 random cards from the shuffled deck. Both players will be assigned 5 random cards each from the shuffled deck. The players' and their opponents' hand and herd sizes are also updated accordingly. After the initial game setup is done, the player turns can begin.

The "options" function will call the "get options" function to check which options can be performed by the current player. Then, according to the selected option, the relevant function will be called to perform the selected game action. In total, there are four actions for the player to choose from; take 1 card from the marketplace, take all the camels, sell cards, and trade between 2 to 5 cards from the marketplace with cards from the player's hand and/or herd.

After each player's turn, a function called "check" will be

called to check if the round is finished. It does so by checking if 3 or more card type tokens have finished and if this is the case, the function will return False to signify that the round is ended.

The "final" function is called when the round is finished. It checks which player has the most Camel cards and assigns this player the Camel bonus token. Then, it will check which player won the game/round by checking who has the most total points. This function will also increment the winner's Seal of Excellence count.

Whenever a card is being dealt to the marketplace from the deck, a check is performed to make sure that the deck is not empty and if so, as per the rules of Jaipur, the game will be finished.

4.2 Objective 2: Develop a reinforcement learning environment to be able to test different algorithms

The second objective of this project is to develop a reinforcement learning environment. This has been accomplished with the use of the OpenAI Gym (<https://openai.com/>) which is a modern toolkit that helps in implementing a reinforcement learning environment as it allows the testing of different algorithms. This environment consists of a class with three functions; init, step and reset.

The "init" function initializes the environment which, in this case, is the initial setup of the game of Jaipur by calling the Jaipur class. It also calls the "round" function from the Jaipur class in order to get the first observation state from the initial environment state.

The "step" function will be called until the round of the game terminates to simulate a full episode. In this implementation,

both players are played with the same agent, however the actions and rewards for each player will be stored separately in order to generate a good policy. Because of this, the "step" function contains a bool variable to signify whose turn it is to play. The current player will choose which action to play and after the action is performed, the "check" function from the Jaipur class is called to check whether the round is finished or not. If the "check" function returns False, the Jaipur class's "final" function is called to see who won the game. All the game actions and their respective rewards of both players will be returned for the policy of the reinforcement learning algorithm to be updated accordingly.

The "reset" function will reset the environment. In this case, the function will be used after every round of the game as one episode is one full round. Therefore, when the episode ends, the game will be reinitialized by calling the Jaipur class.

4.3 Objective 3: Design the observation and action spaces

The third objective is to decide on how to represent the observation space and how to encode the action space. This is challenging due to numerous factors. Firstly, the game has a large action space and therefore, research ([1], [3], [6], [13], [18], [20]) is being performed on how to best go about encoding such a space. Another factor is whether or not the observation space will contain information regarding the opponent's cards and points. Apart from this, there is the consideration of whether the observation space should also contain which cards were sold or were already present in the marketplace. This would be done in order to gain some knowledge on which cards can be pulled from the deck. However, the challenge is to provide the best amount of information about the state whilst also not having a very large observation space. Currently, the observation space is set to represent the player's cards, hand size, herd size and points, as well as the current marketplace cards, the opponent's herd size and the opponent's known points, excluding the points the opponent received from bonus tokens. The current reward function is set as the points given to the player when selling the cards and for having the most Camel cards at the end of the round.

4.4 Objective 4: Evaluate different reinforcement learning algorithms with different reward functions

After the observation and action spaces are finalised, the final objective would be to test different reinforcement learning algorithms in order to figure out which one converges the best. Moreover, the plan is to try out different reward functions with the algorithms as some algorithms may react very differently when given a different reward function. Since the game's environment makes use of a discrete action space, algorithms such as DQN, DDQN and MCTS will be evaluated in the environment.

5. EVALUATION

All the objectives will or have been individually evaluated in order to determine that the objective has been achieved.

5.1 Objective 1: Implement the rules of the game

This objective was first evaluated by testing all the game functions individually with different inputs to verify that the appropriate output was returned for all possible inputs. Checks were performed to make sure that all the values were being modified correctly. Afterwards, the functions were tested together to confirm that the game performs according to the rules. This was achieved by performing random game actions and checking that the output was correct and that the game terminated appropriately, either when the deck was empty, or when 3 or more token types were finished.

5.2 Objective 2: Develop a reinforcement learning environment to be able to test different algorithms

In order to determine that this objective was achieved successfully, testing was performed to make sure that the game works well with the environment. This was conducted by running the needed game functions from the environment and checking that everything was being executed properly. Testing was also performed to make sure that the step function switches correctly between players and terminates when the game round is finished. Further testing will then be performed when the third objective is achieved to make sure that all the functions work as intended with the algorithms.

5.3 Objective 3: Design the observation and action spaces

This objective will firstly be tested with the game alone to make sure that the correct values are being stored and that all the variables are being updated correctly. Afterwards, testing will be performed with the gym environment to make sure that everything is being returned and stored correctly. Finally, when everything is functioning correctly, the true testing will begin by implementing a reinforcement learning algorithm and checking that the action space is being represented in a way that is not difficult for the agent to learn a policy. Changes will possibly be made to the observation and/or action spaces until an ideal solution is found.

5.4 Objective 4: Evaluate different reinforcement learning algorithms with different reward functions

The final objective will be evaluated by performing quantitative and qualitative testing. The testing includes:

- Checking if the policy is improving by seeing if the player's points at the end of each episode are increasing,
- Checking the total points in relation to the number of episodes,
- Checking if the moves that are being carried out by the agent conform to expert strategies by making use of online forums dedicated to the game of Jaipur,
- Checking the number of wins against the published game's AI and potentially even against human players.

6. CONCLUSION

This report provides a detailed description of what was achieved so far, as well as the remaining steps of the proposed solution. The third objective will be achieved by the

end of February. By doing so, there will be enough time for an adequate amount of testing to be performed in order to achieve the final objective by beginning of April. For the full project plan, refer to the Gantt chart found in Appendix A. Certain features of the game of Jaipur were outlined which make the reinforcement learning environment more unique and challenging to train than others. Additionally, an overview of the research carried out thus far about different reinforcement learning methods and algorithms, for two-player score-based games similar to Jaipur, were also presented in this report.

REFERENCES

- [1] Cameron Browne, Dennis J. N. J. Soemers, and Eric Piette. 2021. Strategic Features for General Games. (1 2021). <http://arxiv.org/abs/2101.00843>
- [2] Guillaume Chaslot, Sander Bakkes, Istvan Szita, and Pieter Spronck. Monte-Carlo Tree Search: A New Framework for Game AI. (???). www.aaai.org
- [3] Gabriel Dulac-Arnold, Richard Evans, Hado van Hasselt, Peter Sunehag, Timothy Lillicrap, Jonathan Hunt, Timothy Mann, Theophane Weber, Thomas Degris, and Ben Coppin. 2015. Deep Reinforcement Learning in Large Discrete Action Spaces. (12 2015). <http://arxiv.org/abs/1512.07679>
- [4] Kazuhisa Fujita. 2021. AlphaDDA: Strategies for Adjusting the Playing Strength of a Fully Trained AlphaZero System to a Suitable Human Training Partner. (11 2021). <http://arxiv.org/abs/2111.06266>
- [5] Imran Ghory. 2004. Reinforcement learning in board games. 1 Acknowledgments. (2004).
- [6] Anssi Kanervisto, Christian Scheller, and Ville Hautamäki. 2020. Action Space Shaping in Deep Reinforcement Learning. (4 2020). <http://arxiv.org/abs/2004.00980>
- [7] Wolfgang Konen. 2015. Reinforcement Learning for Board Games: The Temporal Difference Algorithm. (2015). DOI: <http://dx.doi.org/10.13140/RG.2.1.1965.2329>
- [8] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. 2013. Playing Atari with Deep Reinforcement Learning. (12 2013). <http://arxiv.org/abs/1312.5602>
- [9] Luca Pasqualini, Maurizio Parton, Francesco Morandin, Gianluca Amato, Rosa Gini, and Carlo Metta. 2022. Leela Zero Score: a Study of a Score-based AlphaGo Zero. (1 2022). <http://arxiv.org/abs/2201.13176>
- [10] Aske Plaat. 2022. *Deep Reinforcement Learning*. Springer Nature Singapore. DOI: <http://dx.doi.org/10.1007/978-981-19-0638-1>
- [11] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. 2018. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science* 362, 6419 (2018), 1140–1144. DOI: <http://dx.doi.org/10.1126/science.aar6404>
- [12] Richard S. Sutton and Andrew G. Barto. *Reinforcement learning : an introduction*. 526 pages.
- [13] Kristján Tryggvason. 2016. Design and Implementation of a Collectable Trading Card Board Game and Game AI. (2016).
- [14] Hado van Hasselt, Arthur Guez, and David Silver. 2015. Deep Reinforcement Learning with Double Q-learning. (9 2015). <http://arxiv.org/abs/1509.06461>
- [15] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. 2015. Dueling Network Architectures for Deep Reinforcement Learning. (11 2015). <http://arxiv.org/abs/1511.06581>
- [16] Marco Wiering. 2001. Reinforcement Learning in Dynamic Environments using Instantiated Information. (08 2001). https://www.researchgate.net/publication/2393236_Reinforcement_Learning_in_Dynamic_Environments_using_Instatiated_Information
- [17] Marco A Wiering, Jan Peter Patist, and Henk Mannen. 2005. Learning to Play Board Games using Temporal Difference Methods. (2005). www.cs.uu.nl
- [18] Zhiyuan Yao, Tianyu Shi, Site Li, Yiting Xie, Yuanyuan Qin, Xiongjie Xie, Huan Lu, and Yan Zhang. 2022. Towards Modern Card Games with Large-Scale Action Spaces Through Action Representation. (6 2022). <http://arxiv.org/abs/2206.12700>
- [19] Qiyue Yin, Jun Yang, Kaiqi Huang, Meijing Zhao, Wancheng Ni, Bin Liang, Yan Huang, Shu Wu, and Liang Wang. 2021. AI in Human-computer Gaming: Techniques, Challenges and Opportunities. (11 2021). <http://arxiv.org/abs/2111.07631>
- [20] Tom Zahavy, Matan Haroush, Nadav Merlis, Daniel J. Mankowitz, and Shie Mannor. 2018. Learn What Not to Learn: Action Elimination with Deep Reinforcement Learning. (9 2018). <http://arxiv.org/abs/1809.02121>
- [21] Daochen Zha, Jingru Xie, Wenye Ma, Sheng Zhang, Xiangru Lian, Xia Hu, and Ji Liu. 2021. DouZero: Mastering DouDizhu with Self-Play Deep Reinforcement Learning. (6 2021). <http://arxiv.org/abs/2106.06135>

APPENDIX A: PROJECT PLAN

